UNIVERSITY OF GUJRAT

Department of Computer Science

PROJECT DOCUMENTATION

"Daily Core Track"

Smart Daily Life Manager — Console-Based C++ Application

Course Code	CS-103
Subject	Object Oriented Programming
Language	C++ (OOP)
Project Type	Group Project (3 Members)

Group Members	Roll Number
Atiba Younus	25021519-095
Rabia Aman	25021519-038
Maryam Imran	25021519-084

1. Project Overview

Daily Core Track is a console-based application written in C++ that enables students and individuals to manage their complete daily routine in an organized and structured way. The system was built entirely using Object-Oriented Programming (OOP) principles, demonstrating all core OOP concepts including inheritance, polymorphism, encapsulation, abstraction, friend functions, and design patterns.

The application provides six integrated modules: User Profile Management, Task Management, Schedule Planning, Health Tracking, Weekly Activity Analysis, and a Reminder System. All data entered by the user is persisted to plain text files on disk, ensuring information is retained between sessions.

1.1 Project Identity

Project Title	Daily Core Track
Application Type	Console-Based C++ Application
Persistence	Text File I/O (profile.txt, tasks.txt, schedule.txt, health.txt, weekly_report.txt)

1.2 Objectives

- Manage a student's daily routine efficiently and in an organized manner.
- Provide full CRUD (Create, Read, Update, Delete) operations for tasks.
- Plan daily schedules including class timings, study hours, and rest periods.
- Track health metrics such as sleep, water intake, and exercise time.
- Analyze weekly activity patterns and identify the most time-consuming activity.
- Deliver reminders for study sessions and exercise routines.
- Demonstrate all major OOP concepts in a real-world C++ project.

1.3 OOP Concepts Demonstrated

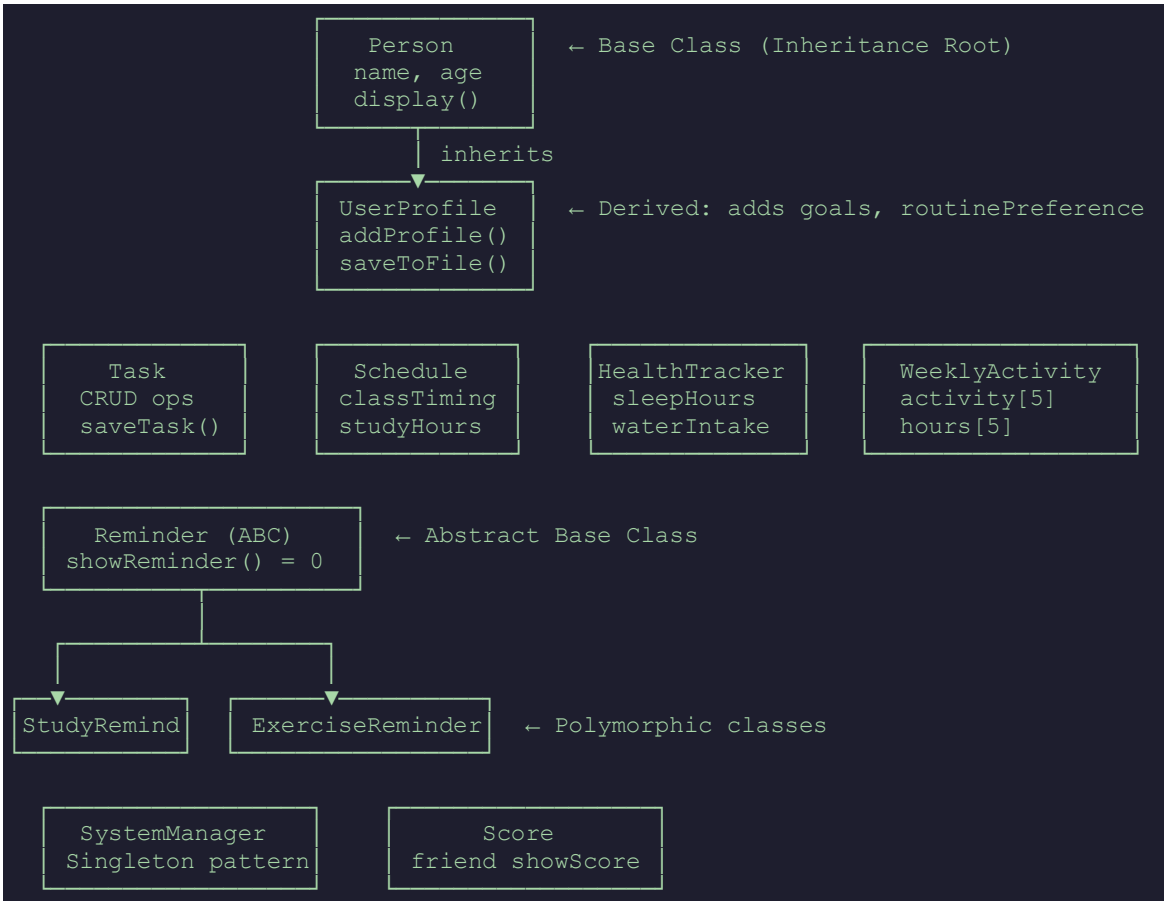
OOP Concept	Implementation in Project
Inheritance	UserProfile extends Person (base class); StudyReminder & ExerciseReminder extend Reminder.
Encapsulation	All class data members are private; accessed only through public methods.
Polymorphism	Reminder* pointers call overridden showReminder() at runtime via virtual dispatch.
Abstraction	Reminder is a pure abstract class (showReminder() = 0) that hides implementation details.

Constructor Overloading	Task and Person classes each define multiple constructors (default + parameterized).
Friend Function	showScore() is a friend of the Score class and directly accesses its private value member.
Singleton Pattern	SystemManager ensures exactly one instance exists via a static getInstance() method.
File Handling	Every module saves and loads data from its own dedicated .txt file using ifstream/ofstream.
Exception Handling	The main menu loop uses try/catch to detect and report out-of-range menu choices.
Virtual Destructor	Person declares a virtual destructor to ensure correct polymorphic cleanup.

2. System Architecture

The application follows a modular, class-based architecture where each functional area of the system is encapsulated inside its own class. All classes are defined in a single C++ source file and interact through the main() function, which drives a menu-based control loop.

2.1 Class Hierarchy Diagram



2.2 File System Layout

All persistent data is stored in plain text files in the same directory as the executable:

File	Owner Module	Stored Fields
`profile.txt`	UserProfile	name, age, goals, routinePreference
`tasks.txt`	Task	taskName, priority, deadline, status
`schedule.txt`	Schedule	classTiming, studyHours, restTime
`health.txt`	HealthTracker	sleepHours, waterIntake, exerciseTime

`weekly_report.txt`	WeeklyActivity	5× (activityName, hoursPerWeek)

2.3 Program Flow

```
Program Start
     |
     ▼
SystemManager::getInstance() → manager->welcome()   [Singleton]
     |
     ▼
Create objects: UserProfile, Task, Schedule, HealthTracker, WeeklyActivity
                StudyReminder, ExerciseReminder
     |
     ▼
┌──────────────────────────────┐
│         MAIN MENU LOOP       │
│   (do-while, exits on choice 0)
│                              │
│  try { if(choice<0||choice>16) │
│          throw choice; }     │
│  catch(int) { error message } │
│                              │
│  switch(choice) → module call │
└──────────────────────────────┘
     |
     ▼
choice == 0 → Program Exits
```

3. Module Documentation

3.1 Base Class: Person

Person is the root base class of the inheritance hierarchy. It stores the two most fundamental attributes shared by any person entity in the system: name and age. It declares a virtual display() method that derived classes can override, and a virtual destructor to ensure safe polymorphic deletion.

Data Members

Member	Type	Description
`name (protected)`	`string`	Full name of the person
`age (protected)`	`int`	Age of the person in years

Methods

Method	Return Type	Description
Person()	–	Default constructor — initialises name to empty string and age to 0.
Person(n, a)	–	Parameterised constructor — sets name and age from arguments.
display()	void	Virtual method that prints name and age to console. Overridden in UserProfile.
~Person()	–	Virtual destructor — ensures derived class destructors are called correctly.

3.2 Module 1: UserProfile

UserProfile inherits publicly from Person and extends it with two additional fields: goals (what the user wants to achieve) and routinePreference (e.g., morning person / night owl). This is the first module a user interacts with. Data is saved to profile.txt and reloaded whenever the user views their profile.

Data Members

Member	Type	Description
`goals (private)`	`string`	User's personal goals, e.g., 'Score 3.8 GPA this semester'
`routinePreference (private)`	`string`	User's preferred routine style, e.g., 'Morning Routine'
`name (inherited)`	`string`	Inherited from Person (protected)

`age (inherited)`	`int`	Inherited from Person (protected)

Methods

Method	Return Type	Description
addProfile()	void	Prompts user to enter name, age, goals, and routinePreference. Calls saveToFile() and confirms success.
display()	void	Overrides Person::display(). Prints all four profile fields with a section header.
saveToFile()	void	Opens profile.txt (overwrite mode) and writes all four fields line by line.
loadFromFile()	void	Reads profile.txt and populates name, age, goals, routinePreference into the object.

File Format — profile.txt

```
Atiba Younus
20
Score 3.8 GPA
Morning Routine
```

3.3 Module 2: Task Management

The Task class is a standalone class (not derived from Person) that provides full CRUD operations for a single task. It stores the task's name, its priority level, a deadline string, and a status field that starts as 'Pending' and can be promoted to 'Completed'. Data is saved to tasks.txt.

Note: The current implementation manages one task at a time. Calling addTask() or deleteTask() replaces the entire tasks.txt file. This is by design for the current version of the project.

Data Members

Member	Type	Description
`taskName`	`string`	Name/title of the task
`priority`	`string`	Priority level, e.g., 'High', 'Medium', 'Low'
`deadline`	`string`	Deadline date as a string, e.g., '2025-06-15'
`status`	`string`	Current state of the task: 'Pending' or 'Completed'

Methods

Method	Return Type	Description
addTask()	void	Collects taskName, priority, deadline from user. Sets status='Pending'. Calls saveTask().
displayTask()	void	Prints all four task fields with a section header.
updateTask()	void	Prompts for new values of all three editable fields and calls saveTask().
deleteTask()	void	Clears all member variables and overwrites tasks.txt with an empty file.
markCompleted()	void	Loads current task, sets status='Completed', and calls saveTask() to persist the change.
saveTask()	void	Opens tasks.txt (overwrite) and writes all four fields line by line.
loadTask()	void	Reads tasks.txt into the object's four data members.

3.4 Module 3: Schedule Planner

The Schedule class stores a student's daily timetable across three fields: class timings, study hours, and rest time. These are stored as free-form strings, giving users flexibility in how they describe their schedule. Data is persisted in schedule.txt.

Data Members

Member	Type	Description
`classTiming`	`string`	Description of class schedule, e.g., '8am-12pm'
`studyHours`	`string`	Planned study hours, e.g., '3 hours after dinner'
`restTime`	`string`	Planned rest or sleep time, e.g., '10pm-6am'

Methods

Method	Return Type	Description
addSchedule()	void	Prompts for all three timing fields and calls saveSchedule().
displaySchedule()	void	Prints classTiming, studyHours, and restTime with a section header.
saveSchedule()	void	Writes three fields to schedule.txt, one per line.
loadSchedule()	void	Reads schedule.txt into the three data members.

3.5 Module 4: Health Tracker

HealthTracker records three daily health metrics as integers: sleep hours, glasses of water consumed, and minutes of exercise. The module encourages students to maintain healthy habits by providing a daily health report. Data is saved to health.txt.

Data Members

Member	Type	Description
`sleepHours`	`int`	Total hours of sleep recorded for the day
`waterIntake`	`int`	Number of glasses of water consumed
`exerciseTime`	`int`	Exercise duration in minutes

Methods

Method	Return Type	Description
HealthTracker()	–	Default constructor — initialises all three metrics to 0.
addHealthData()	void	Reads sleepHours, waterIntake, exerciseTime from user and calls saveHealth().
displayHealthReport()	void	Prints a formatted health report showing all three metrics.
saveHealth()	void	Writes the three integer values to health.txt, one per line.
loadHealth()	void	Reads the three integer values from health.txt using integer extraction (>>).

3.6 Module 5: Weekly Activity Analyzer

WeeklyActivity is the most analytically capable module. It tracks exactly five activities per week, storing their names and total hours in two parallel arrays. It can display the full weekly breakdown and automatically identify which activity consumed the most time by running a simple linear search for the maximum.

Data Members

Member	Type	Description
`activity[5]`	`string[]`	Array of 5 activity name strings
`hours[5]`	`int[]`	Parallel array of hours spent on each activity

Methods

Method	Return Type	Description
WeeklyActivity()	–	Default constructor — initialises all 5 activity strings to empty and all hours to 0.
addActivities()	void	Loops 5 times to collect activity name and hours per activity. Calls saveReport().
displayAllActivities()	void	Iterates and prints all 5 activities with their hours in a weekly report format.
showMostActiveTask()	void	Linear scan of hours[] to find the maximum value and prints that activity's name and hours.
saveReport()	void	Writes 5 pairs (activityName, hours) to weekly_report.txt — 10 lines total.
loadReport()	void	Reads 5 pairs from weekly_report.txt back into the parallel arrays.

4. OOP Concept Deep Dives

4.1 Abstraction & Polymorphism — Reminder System

The Reminder class is a pure abstract class (also called an Abstract Base Class, or ABC). It declares showReminder() as a pure virtual function (= 0), which means no object of type Reminder can ever be created directly. Its sole purpose is to define a contract: any class that inherits from Reminder must provide its own implementation of showReminder().

Two concrete classes implement this contract: StudyReminder and ExerciseReminder. In main(), base class pointers (Reminder* r1 and Reminder* r2) are assigned to the addresses of these concrete objects. When r1->showReminder() and r2->showReminder() are called, C++ uses the virtual dispatch table (vtable) to call the correct version at runtime — this is runtime polymorphism.

```
Reminder *r1 = &sr;   // Pointer of base type → object of derived type
Reminder *r2 = &er;

r1->showReminder();   // Calls StudyReminder::showReminder()  → vtable lookup
r2->showReminder();   // Calls ExerciseReminder::showReminder() → vtable lookup
```

4.2 Singleton Design Pattern — SystemManager

SystemManager implements the Singleton pattern, which guarantees that only one instance of the class can ever exist in the entire program. This is achieved through three mechanisms:

- The constructor is private — so no code outside the class can create an instance with 'new SystemManager()'.
- A static pointer 'instance' (initialised to NULL) holds the one and only object once it is created.
- The public static method getInstance() checks if instance is NULL. If it is, it creates the object. Either way, it returns the existing pointer.

```
// First call: instance is NULL → creates the object
SystemManager *manager = SystemManager::getInstance();

// Any subsequent call: instance is not NULL → returns existing object
// → Always the same single object throughout the program
manager->welcome();  // Prints the application banner
```

4.3 Friend Function — Score & showScore()

The Score class stores a private integer value representing a productivity score. Normally, no function outside the class can access private members. However, the showScore() function is declared as a friend of Score, which grants it special access to read value directly (without needing a getter method).

In main() under menu option 16, a Score object with value 95 is created and passed to showScore(), which then prints the productivity score to the console. This demonstrates the friend function concept: a non-member function with privileged access to a class's private data.

```
Score s(95);
showScore(s);   // Prints: Productivity Score: 95
// showScore is NOT a member of Score — it is a standalone function
// but can access s.value because it is declared 'friend' in Score
```

4.4 Exception Handling — Menu Validation

The main menu loop is wrapped in a try/catch block to handle invalid user input gracefully. Before the switch statement executes, the code checks whether the entered choice is outside the valid range (0–16). If it is, the integer value is thrown as an exception.

The catch(int) block intercepts this exception and prints an error message instead of crashing or entering the switch with an invalid value. This demonstrates structured exception handling in C++ as an alternative to simple if/else guards.

```
try {
    if (choice < 0 || choice > 16)
        throw choice;           // Throw the invalid integer
    switch(choice) { ... }
}
catch (int) {
    cout <<"Exception Caught: Invalid Menu Choice!"<< endl;
}
```

4.5 Inheritance — Person → UserProfile

UserProfile derives from Person using public inheritance. This means all public and protected members of Person (name, age, display()) are accessible inside UserProfile. UserProfile adds two private members of its own: goals and routinePreference. It also overrides display() to print the extended profile, and calls Person::display() inside its override to reuse the parent's output logic.

```
class UserProfile : public Person {
    // Inherits: name (protected), age (protected), display() (virtual)
    // Adds:     goals (private), routinePreference (private)

    void display() override {
        Person::display();      // Call parent version first
        cout << goals << endl;  // Then add extra fields
        cout << routinePreference << endl;
    }
};
```

5. Menu System Reference

The application runs a do-while loop that displays 17 options (0–16). The loop continues until the user enters 0. All input validation is handled via the try/catch block described in Section 4.4.

Choice	Module	Action
1	**User Profile**	Add a new user profile (name, age, goals, routine preference)
2	**User Profile**	Load profile from file and display it
3	**Task Management**	Add a new task (name, priority, deadline)
4	**Task Management**	Load and display the current task
5	**Task Management**	Update all task fields
6	**Task Management**	Delete the task (clears file)
7	**Task Management**	Load task and mark it as Completed
8	**Schedule Planner**	Add daily schedule (class timing, study hours, rest time)
9	**Schedule Planner**	Load and display the schedule
10	**Health Tracker**	Enter today's health data
11	**Health Tracker**	Load and display the health report
12	**Reminder System**	Display study and exercise reminders via polymorphism
13	**Weekly Activity**	Enter 5 weekly activities with hours
14	**Weekly Activity**	Load and display the full weekly report
15	**Weekly Activity**	Show the most time-consuming activity
16	**Productivity Score**	Display a hardcoded productivity score (95) via friend function
0	**System**	Exit the application

6. Conclusion

Daily Core Track successfully demonstrates all major Object-Oriented Programming concepts required for a useful student management application. The system is organized into clean, well-defined modules, each encapsulated within its own class, with clear responsibilities and well-named methods.

The vision document described five modules;

The implemented code delivers six modules (the additional Weekly Activity Analyzer) along with supplementary classes (Score, SystemManager) that showcase advanced OOP concepts including the Singleton design pattern and friend functions.

The project provides a solid foundation that can be extended in future iterations with features such as an array of tasks (instead of a single task), user authentication, deadline-based automatic alerts, and a graphical interface. As built, it is a complete and functional demonstration of C++ OOP principles.

COMPLETED